QALight

Frontend-розробка

Заняття 5. Каскадні стилі сторінок

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Синтаксис CSS. Селектори. Класи та ідентифікатори. Глобальний стиль документа. Практика стилізації. Нормалізація. Простір імен. Правила @.

Синтаксис CSS

Селектори. Класи та ідентифікатори. Глобальний стиль документа. Практика стилізації. Нормалізація. Простір імен. Правила @.

CSS

Теорія: css

CSS - це акронім **Cascading Style Sheets/Каскадні таблиці стилів** - формальна мова опису зовнішнього вигляду документа, що створений за допомогою мови розмітки. **HTML** використовується для структурування змісту сторінки. **CSS** - це **засіб оформлення зовнішнього вигляду веб-сторінок**, що створені за допомогою HTML, XHTML, XML, SVG або XUL.

Основна мета **CSS** полягає в тому, щоб дозволити двигуну браузера малювати елементи сторінки за допомогою певних функцій, таких як кольори, позиціонування або прикраси. CSS містить правила стилю, що інтерпретуються браузером, і зрештою застосовуються до відповідних елементів документа. Правило стиля складається з трьох частин:

- **Селектор** - це компоненти CSS, які дозволяють точно націлювати та застосовувати стилі до певних елементів HTML на веб-сторінці. Вони визначають, на які елементи HTML-документа впливатимуть визначені правила CSS.
- **Властивість -** ідентифікатор, що визначає, яка функція розглядається.
- **Значення**, яке описує, як механізм має обробляти цю функцію. Кожна властивість має набір дійсних значень, визначених формальною граматикою, а також семантичне значення, реалізоване механізмом браузера.

Глобальні атрибути

Теорія: css

Глобальні атрибути - це атрибути, загальні **для всіх елементів HTML**; їх можна використовувати для всіх елементів, хоча вони можуть не впливати на деякі елементи.

Глобальні атрибути можуть бути вказані для всіх елементів HTML, навіть для тих, які не вказані в стандарті. Це означає, що будь-які нестандартні елементи все одно повинні дозволяти ці атрибути, навіть якщо використання цих елементів означає, що документ більше не сумісний з HTML5. Наприклад, браузери, сумісні з HTML5, приховують вміст, позначений як **<foo hidden>…</foo>**, навіть якщо <foo> не є дійсним елементом HTML.

Окрім основних глобальних атрибутів HTML, також існують такі глобальні атрибути:

- **xml:lang і xml:base** - вони успадковані від специфікацій XHTML і застаріли, але зберігаються для цілей сумісності.
- Атрибут ролі **ARIA** та численні стани та властивості aria-* використовуються для забезпечення доступності.
- Атрибути обробника подій **on<event>**

Ідентифікатори

Теорія: Ідентифікатори

Ідентифікатор (ID) призначений для ідентифікації елемента при створенні посилань на нього, виконанні скриптів чи застосування стилей. **ID має бути унікальним у межах документа**.

Значення ідентифікатора ID не повинно містити пробілів. Браузери розглядають невідповідні ідентифікатори, які містять пробіли, так, ніби пробіли є частиною ідентифікатора. Технічно значення атрибута id може містити будь-який символ, окрім пробілів. Однак, щоб уникнути помилок, слід використовувати **лише літери ASCII, цифри, «_» та «-»**, а значення атрибута id **має починатися з літери**.

Ідентифікатор, що починається з цифри (наприклад, 1234-322-678) або дефіса, після якого йде цифра (наприклад, -123), хоча й дійсний у HTML, може призвести до проблем під час використання в CSS, JavaScript і веб-API : Якщо ідентифікатор починається з цифри або одного дефіса, після якого відразу йде цифра, і дефіс, і цифра повинні бути екрановані в CSS. Наприклад, хоча id="544-383-3388" та id="-3Pi" дійсні в HTML, ідентифікатори мають бути екрановані. Елемент із цими значеннями ідентифікатора можна націлити в CSS за допомогою #\35 44-383-3388 і #\2D \33 pi.

Класи

Теорія: css

Глобальний атрибут **class** - це розділений пробілами список чутливих до регістру класів елемента. Класи дозволяють CSS і JavaScript отримувати доступ до певних елементів за допомогою селекторів класів або функцій, таких як метод DOM document.getElementsByClassName.

```
<p>Narrator: This is the beginning of the play.</p>
<p class="note editorial">Above point sounds a bit obvious. Remove/rewrite?</p>
<p>Narrator: I must warn you now folks that this beginning is very exciting.</p>
<p class="note">[Lights go up and wind blows; Caspian enters stage right]</p>
```

Хоча специфікація не висуває вимог до імен класів, веб-розробникам **рекомендується** використовувати **імена, які описують семантичну мету елемента**, а не презентацію елемента. Семантичні назви залишаються логічними, навіть якщо змінюється оформлення сторінки.

Значення **class** та **id** мають містити лише літери, цифри, дефіс та нижнє підкреслення. Вони **мають починатись лише з літери**. **Імена класів та ідентифікаторів**, на відміну від назв тегів та їх атрибутів, **чутливі до регістру**.

Правила побудови CSS

Теорія: css

Пара властивостей і значень називається **оголошенням**, і будь-який механізм CSS обчислює, які оголошення застосовуються до кожного окремого елемента сторінки, щоб відповідним чином розмістити його та стилізувати. І **властивості, і значення за замовчуванням** у CSS **не враховують регістр**. Пара розділяється двокрапкою **«:»** (пробіли перед, між і після властивостей і значень, але не обов'язково всередині, ігноруються.

У CSS є сотні різних властивостей і практично нескінченна кількість різних значень. Не всі пари властивостей і значень дозволені, і кожна властивість визначає дійсні значення. Якщо значення недійсне для даної властивості, оголошення вважається недійсним і повністю ігнорується механізмом CSS.

Оголошення групуються в **блоки**, тобто в структуру, розділену відкриваючою дужкою '{' (**ЛІВА ФІГУРНА ДУЖКА**) і закриваючою '}' (**ПРАВА ФІГУРНА ДУЖКА**). Блоки іноді можуть бути вкладеними, тому відкриваючі та закриваючі фігурні дужки повинні відповідати структурі вкладених блоків.

Блоки оголошень

Теорія: css

Блоки оголошень оголошення може бути порожнім, тобто містити нульове оголошення. Пробіли навколо декларацій ігноруються. Останнє оголошення блоку не обов'язково завершувати крапкою з комою. Вміст блоку оголошень можна помістити в **атрибут style тега**.

Твердження або **Інструкція** - це будівельний блок, який починається будь-якими символами, що не є пробілами, і закінчується першою закриваючою дужкою або крапкою з комою та не входить до іншої пари {}, () або []. Існує два види тверджень:

1. **Набори правил**, які, пов'язують набір оголошень CSS з умовою, описаною селектором.
2. **Правила at**, які починаються зі знака at, '**@**', після якого йде ідентифікатор. Кожен тип **at-правил, визначених ідентифікатором**, може мати власний внутрішній синтаксис і семантику. Вони використовуються для передачі метаданих (наприклад, @charset або @import), умовної інформації (наприклад, @media або @document) або описової інформації (наприклад, @font-face).

Будь-який оператор, який не є набором правил або правилом at, є недійсним і ігнорується.

Набори правил

Теорія: css

CSS дозволяє застосовувати різні декларацій до різних частин документа, пов’язуючи **умови з блоками оголошень**. Перед кожним **блоком оголошень** стоїть один або більше селекторів, **розділених комами**, які є умовами вибору деяких елементів сторінки. Список селектора та пов’ язаний блок оголошень разом називаються **набором правил** або правилом.

Набори правил - це основні будівельні блоки таблиці стилів, яка часто складається з великого списку. Але є й інша інформація, яку веб-автор хоче передати в таблиці стилів, як-от набір символів, інші зовнішні таблиці стилів для імпорту, накреслення шрифту чи описи лічильників списку та багато іншого. Для цього він використовуватиме інші типи операторів.

Коментарі /* ... */ дозволяють документувати логіку та структуру селекторів.

```
div {
  width: 200px; /* Ширина блоку */
  margin: 10px; /* Поля  павколо елементу */
  float: left; /* … */
}
```

Змінні

Теорія: css

Змінні – це текстові рядки, які можна використовувати в програмах і можуть бути змінені відповідно до потреб програміста. **Змінні CSS** – це змінні, які можна використовувати в таблицях стилів CSS.

Змінні можна використовувати **локально** і **глобально** залежно від того, як вони визначені.

Якщо визначити змінну CSS за допомогою **кореневого** елемента, можна використовувати цю змінну **глобально**, проте якщо визначити змінну для певного елемента, цю змінну можна використовувати лише в коді стилю цього елемента.

Щоб оголосити змінну в CSS, використовують такий формат: --nameVar:

```
body {
    --mainColor: limegreen;
}
```

Щоб використати змінну в CSS, застосовують функцію **var()**:

```
div {
    color: var(--mainColor);
}
```

Області видимості змінних

Теорія: css

Місце в ієрархії CSS, де оголошено змінну CSS, визначатиме її рівень видимості на нижчих рівнях ієрархії. Змінна CSS, яка використовується на всій сторінці, оголошується в HTML-елементі **html**. Змінні CSS, оголошені в HTML-елементі, будуть видимі для споживання його дочірніми елементами. Як правило, **змінна CSS видима лише для дочірніх елементів батьківського елемента, у якому вона оголошена**. Елемент body є батьківським для своїх дочірніх елементів div, Div 1 і Div 2

```
<body>
    <div>Div 1</div>
    <div>Div 2</div>
</body>
```

Змінна CSS mainColor оголошується в елементі body і використовується в div.

```
body {
    --mainColor: limegreen;
}
div {
    color: var(--mainColor);
}
```

Глобальні області видимості

Практика: css

Кажуть, що змінні CSS, оголошені в елементі html, знаходяться в **глобальній області видимості.** Це означає, що до них можна отримати доступ будь-де в CSS. HTML-елементі html додається до кореня документа. Усі елементи у веб-документі знаходяться в **кореневому** `елементі`, тому змінні CSS, оголошені в HTML-елементі **html**, каскадно опускатимуться до всіх рівнів документа.

Крім того, оголошені **глобальні** змінні CSS будуть видимі для всіх елементів документа.

Змінні CSS також можна оголошувати **на всіх рівнях ієрархії CSS або окремому селектор**і. Змінні, оголошені на цих рівнях, видимі або обмежені **локально** для селектора та його дочірніх вузлів.

Змінні CSS можна **підняти**. Це означає, що змінні CSS можна використовувати до того, як вони будуть оголошені. Змінна CSS --bgColor використовувалася до того, як її було оголошено в HTML-елементі html. Таким чином, **змінні CSS можуть бути доступні спочатку, а потім оголошені**. Це робить змінні CSS дуже потужною функцією.

```
body {  background-color: var(--bgColor);  }

html {  --bgColor: rgb(221, 221, 221);   }
```

Способи підключення CSS

Практика: css

Вбудовані стилі – це оголошення CSS, які впливають на один елемент HTML, що міститься в атрибуті **style**. Реалізація вбудованого стилю в документі HTML може виглядати так:

```
<h1 style="color: blue;background-color: yellow;border: 1px solid black;">
    Hello World!
</h1>
<p style="color:red;">This is my first CSS example</p>
```

Уникайте використання CSS таким чином, коли це можливо. Це протилежність передовій практиці. По-перше, це найменш ефективна реалізація CSS. Одна зміна стилю може вимагати кількох редагувань на одній веб-сторінці. По-друге, вбудований CSS також змішує презентаційний код (CSS) з HTML і вмістом, що ускладнює читання та розуміння. Розділення коду та вмісту полегшує обслуговування для всіх, хто працює на веб-сайті.

Елемент <style>

Практика: css

Елемент **<style>** містить інформацію про стиль для документа або частини документа. Він містить CSS, який застосовується до вмісту документа, що містить елемент <style>.

Елемент **<style> повинен міститися всередині <head>** документа. Загалом, краще розміщувати свої стилі у зовнішніх таблицях стилів і застосовувати їх за допомогою елементів <link>.

```
<style>
    body {
        color: red;
    }
</style>
```

Якщо ви включите кілька елементів <style> і <link> у свій документ, вони будуть **застосовані до документу у тому порядку, в якому вони включені** в документ - переконайтеся, що ви включили їх у правильному порядку, щоб уникнути неочікуваних каскадних проблем.

Елемент <link>

Практика: css

Елемент **<link>** визначає зв'язки між поточним документом і зовнішнім ресурсом. Цей елемент найчастіше використовується для посилань на таблиці стилів, але також використовується для встановлення піктограм сайтів (як піктограм стилю «favicon», так і піктограм для головного екрана та програм на мобільних пристроях), серед іншого.

Щоб зв’язати зовнішню таблицю стилів, ви повинні включити елемент <link> у свій <head> таким чином:

```
<link href="main.css" rel="stylesheet" />
```

Атрибут **rel** означає «відносини» і є однією з ключових особливостей елемента <link> - значення вказує на те, як пов’язаний елемент із документом, що містить.

Значення атрибута **rel=**stylesheet дійсне для елемента <link>, він **імпортує зовнішній ресурс для використання як таблицю стилів**. Атрибут type не потрібен, оскільки це таблиця стилів text/css, що є значенням за замовчуванням.

Атрибут rel

Практика: css

Атрибут rel визначає зв'язок між зв'язаним ресурсом і поточним документом. Дійсний для **<link>, <a>, <area> і <form>**, можливі значення залежать від елемента, де встановлено атрибут.

Тип зв’язків визначається значенням атрибута rel, який, якщо присутній, повинен мати значення, яке є невпорядкованим набором унікальних ключових слів, розділених пробілами. На відміну від імені класу, яке не виражає семантику, атрибут **rel повинен виражати лексеми**, які семантично дійсні як для машин, так і для людей. Поточні реєстри для можливих значень атрибута rel - це реєстр зв’язків посилань IANA, стандарт HTML Living Standard і сторінка існуючих значень rel у вікі мікроформатів, як запропоновано стандартом Living Standard. Якщо використовується атрибут rel, якого немає в одному з трьох джерел вище, деякі засоби перевірки HTML (такі як Служба перевірки розмітки W3C) створять попередження.

Атрибут rel стосується елементів **<link>, <a>, <area> і <form>**, але деякі значення мають відношення лише до підмножини цих елементів. Як і всі значення атрибутів ключових слів HTML, ці **значення нечутливі до регістру.**

Атрибут rel

Практика: css

Атрибут **rel не має значення за умовчанням**. Якщо атрибут пропущений або якщо жодне зі значень в атрибуті не підтримується, тоді документ не має певного зв'язку з цільовим ресурсом, крім наявності гіперпосилання, тоді **елемент не створює жодних посилань**. <a> і <area> залишаться посиланнями, але **без визначеного зв'язку**.

Якщо атрибут **rel** використовується з ключовим словом **alternate**, воно визначає альтернативну таблицю стилів. У цьому випадку додайте непорожню назву. Атрибут rel, доданий до <link rel="alternate stylesheet">, створює **альтернативну таблицю стилів**. Під час визначення альтернативної таблиці стилів за допомогою <link rel="alternate"> атрибут title є обов'язковим.

```
<link rel="stylesheet" href="default.css" /> <!-- a persistent style sheet -->
<link  rel="alternate stylesheet"  href="highcontrast.css"  title="High contrast" /><!-- alternate style sheets -->
```

Ви також можете вказати тип медіа або запит всередині атрибута медіа; тоді цей ресурс буде завантажено, лише якщо умова медіа виконується. Наприклад:

```
<link href="print.css" rel="stylesheet" media="print" />
<link href="mobile.css" rel="stylesheet" media="screen and (max-width: 600px)" />
```

Регулярні правила

Практика: css

At-rules - це оператори CSS, які вказують CSS, як поводитись. Вони починаються зі знака at, **'@' (COMMERCIAL AT)**, після якого йде ідентифікатор і включає все до наступної крапки з комою, ';' (КРАПКА З КОМА), або наступного блоку CSS, залежно від того, що відбудеться раніше.

Існує кілька регулярних правил at кожне з яких має інший синтаксис:

@charset - визначає набір символів, який використовується таблицею стилів.

@import - повідомляє механізму CSS включити зовнішню таблицю стилів.

@namespace - повідомляє механізму CSS, що весь його вміст повинен вважатися префіксом простору імен XML.

```
/* Загальна структура */

@identifier (RULE);

/* Приклад: вказує браузеру використовувати набір символів UTF-8 */

@charset "utf-8";
```

Правило @import

Практика: css

Правило **@impor**t CSS використовується для **імпорту правил стилю** з інших таблиць стилів. Правило @import **має бути визначене у верхній частині таблиці стилів перед будь-яким іншим правилом at** (крім @charset і @layer) і оголошеннями стилів, інакше воно буде проігноровано.

```
@import url;
@import url layer;
@import url layer(layer-name);
@import url layer(layer-name) list-of-media-queries;
@import url supports(supports-condition);
@import url list-of-media-queries;
```

Де **url** є рядком або функцією **url()**, що посилається на місцезнаходження ресурсу для імпорту. URL може бути абсолютним або відносним. **Правила @import мають передувати всім іншим типам правил**, окрім правил @charset і операторів @layer для створення шару.

```
@import url("my-imported-styles.css");
/* more styles */
```

Умови імпорту залежно від медіа

Практика: css

Щоб агенти користувачів могли уникнути отримання ресурсів **для непідтримуваних типів медіа**, автори можуть вказати **умови імпорту залежно від медіа**. Цей умовний імпорт визначає медіа-запити, розділені комами, після URL-адреси.

```
@import url("fineprint.css") print;
@import url("bluish.css") print, screen;
@import "common.css" screen;
```

За відсутності медіа-запиту імпорт не залежить від використовуваного медіа. Вказівка **all** для списку медіа-запитів має той самий ефект.

Агенти користувача можуть використовувати функцію **supports()** у правилі @import, щоб отримати ресурси, лише якщо певний набір функцій підтримується (або не підтримується).

```
@import url("gridy.css") supports(display: grid) screen and (max-width: 400px);
@import url("flexy.css") supports(not (display: grid) and (display: flex)) screen
and (max-width: 400px);
```

Функція url()

Практика: css

Функція **url()** CSS використовується для включення файлу. Параметр функції - це **абсолютна URL-адреса, відносна URL-адреса, URL-адреса blob-файлу або URL-адреса даних**.
Функцію url() можна передати як параметр інших функцій CSS, як-от функцію attr().

URL-адресою ресурсу може бути зображення, шрифт або таблиця стилів.

Існує різниця між URI та URL. **URI** ідентифікує ресурс. **URL-адреса** – це тип URI, який описує розташування ресурсу. URI може бути URL-адресою або назвою (URN) ресурсу.
Функція url() позначає лише справжні <url>.

```
/* Simple usage */
url(https://example.com/images/myImg.jpg);
url(…);
url(myFont.woff);
url(#IDofSVGpath);
```

Універсальний селектор

Практика: css

Селектори CSS - це компоненти CSS, які дозволяють застосовувати стилі до певних елементів HTML. Вони визначають, на які елементи HTML-документа впливатимуть визначені правила CSS. Загалом, селектори вибирають елемент HTML і застосовують стиль до нього.

- Базові селектори
- Селектор елементів
- Селектор класу
- Селектор id
- Універсальний селектор

Універсальний селектор * виділяє всі елементи на сторінці. Щоб виділити всі елементи веб-сторінки,:

```
* {
    margin: 0;
    padding: 0;
}
```

Універсальний селектор відповідає будь-якому елементу.

Селектори елементів

Практика: css

Селектор елементів вибирає елементи на основі назви тегу. Селектор елемента застосовує формат для усіх елементів даного типу на веб-сторінці. Наприклад,

```
H1 {   font-size: 2.2rem;}
```

задає загальний стиль форматування усіх заголовків h1.

```
p {    font-family: arial, helvetica, sans-serif;}
```

задає загальний стиль форматування усіх p.

Селектор елемента застосовує формат для усіх елементів даного типу на веб-сторінці. Наприклад,

```
H1 {   font-size: 2.2rem;}
```

задає загальний стиль форматування усіх заголовків h1.

```
p {
   font-family: arial, helvetica, sans-serif;
}
```

задає загальний стиль форматування усіх p.

Селектор класів

Практика: css

Щоб вибрати елементи з їх іменами класів, використовуйте . безпосередньо перед назвою класу. Селектори класів дозволяють задавати стилі для одного чи більше елементів з однаковим іменем класу.
Наприклад, для створення заголовка з класом .h2 необхідно добавитти атрибут **class** зі значенням .h2 до початкового тегу **<h2>** та задати стилі для вказаного класу.
Стилі, що створені за допомогою класу, можна застосувати до інших елементів

```
.h2 {
  font-size: 1.9rem;
}
```

Селектори класів дозволяють задавати стилі для одного чи більше елементів з однаковим іменем класу.
Наприклад, для створення заголовка з класом .h2 необхідно добавитти атрибут **class** зі значенням .h2 до початкового тегу **<h2>** та задати стилі для вказаного класу.
Стилі, що створені за допомогою класу, можна застосувати до інших елементів

```
.h2 {
  font-size: 1.9rem;
```

Селектор id

Практика: css

Оскільки ідентифікатор елемента є унікальним на сторінці, селектор ідентифікатора вибирає один елемент із певним атрибутом ідентифікатора. Щоб вибрати елемент із його конкретним ідентифікатором, використовуйте символ **#** перед назвою ідентифікатора.

```
#main {
    width: 100%;
    color: #555;
}
```

Селектор ідентифікатора форматує один конкретний елемент. Ідентифікатор **id має бути унікальним на веб-сторінці**.

```
<main role="main" class="container" id='main'>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Accusantium saepe placeat
    quisquam, eveniet aliquam ipsum amet cupiditate repudiandae obcaecati, impedit aperiam.
</main>
```

Комбінатори

Практика: css

Є такі види комбінаторів:

- Комбінатор нащадків
- Дочірній комбінатор
- Комбінатор наступного рівня
- Загальний комбінатор братів і сестер

Нащадок - це просто елемент, який знаходиться всередині іншого елемента. Наприклад:

```
<div>
    <h1>This is a heading.</h1>
</div>
Тут <h1> є нащадком <div>.
```

Комбінатор нащадка

Практика: css

Комбінатор нащадків (**descendent combinator**) вибирає елементи, які є нащадками зазначеного батьківського елемента. Ключова ідея тут полягає в тому, що не треба турбуватися про те, наскільки глибоко вкладені елементи-нащадки всередині батьківського. Незалежно від того, чи знаходяться вони безпосередньо в батьківській системі, чи на кількох рівнях, можна вибрати їх всі. Наприклад:

```
<section>
   <p>This is a paragraph.</p>
   <div>
     <p>This is another paragraph.</p>
   </div>
</section>
```

Щоб вибрати нащадка, напишіть назву тегу, атрибут класу або атрибут id батьківського елемента, а після пробілу напишіть ім’я нащадка.

```
section p {
 font-size: 20px;
}
```

Дочірній комбінатор

Практика: css

Дочірній комбінатор вибирає елементи, які є **прямими дочірніми елементами зазначеного батьківського елемента**.

```
<ul>
   <li>HTML</li>
   <li>CSS</li>
</ul>
```

Щоб вибрати прямий дочірній елемент, використовуйте знак > між батьківським і дочірнім елементами.

```
ul > li {
   font-weight: bold;
}
```

Комбінатор наступного рівня

Практика: css

Комбінатор **наступного рівня** (сусіднього брата - **Next-sibling** +) вибирає елемент, який є **безпосереднім братом зазначеного елемента**, і між ними не повинно бути інших елементів .

```
/* Пробіл навкруг комбінатора + не є обов'язковим, проте рекомендується. */
former_element + target_element { style properties }
```

```
li:first-of-type + li {
      color: red;
}
<ul>
 <li>One</li>
 <li>Two!</li>
 <li>Three</li>
</ul>
```

Загальний однорідний комбінатор

Практика: css

Загальний однорідний комбінатор вибирає однорідні елементи, які мають один і той самий батьківський елемент і розташовані після зазначеного елемента. **Загальний однорідний комбінатор представлений символом ~.**

```
<p>I'm sibling paragraph of div.</p>
<div>
   <p>This is a paragraph inside a div.</p>
   <p>This is another paragraph inside a div.</p>
</div>
<p>I'm sibling paragraph of div.</p>
<h6>I'm sibling heading of div.</h6>
<p>I'm sibling paragraph of div.</p>
```

Загальний однорідний комбінатор вибирає однорідні елементи, які йдуть після зазначеного елемента.

```
div ~ p {
    color: red;
}
```

Селектор групування

Практика: css

Селектор групування (,) дозволяє застосовувати однакові стилі до кількох елементів.

```
h1, h2, h3 {
    font-family:  'Arial', sans-serif;
}
```

Встановлює узгоджений шрифт на трьох рівнях заголовків. Підхід, що економить час, коли різні елементи **мають спільний набір стилів**.

Вкладені селектори

Практика: css

Вкладені селектори застосовуватимуться **лише в контексті батьківського селектора**.

```
    <ul class="nav">
       <li class="nav-item">
         <a href="#" class="nav-link"><span>Home</span></a>
       </li>
```

Використання вкладеності також покращує читабельність, запобігає повторенню та може зменшити загальний розмір вашого CSS.

```
.nav {
   min-width: 16em;
   & li {
       list-style: none;
       & a {
           background-color: transparent;
           color: inherit;
           height: 3.25rem;
            text-decoration: none;
       }
   }
}
```

Normalize.css

Практика: css

Використання Normalize.css може допомогти вирішити поширену проблему, з якою стикаються багато розробників: невідповідності в стилях за замовчуванням елементів HTML у різних браузерах. Скидаючи стандартні стилі елементів, Normalize.css допомагає гарантувати, що стилі, застосовані до елементів HTML, узгоджені в усіх браузерах.

Однією з головних переваг використання CDN (**Content delivery network**) Normalize.css у проектах є те, що це може допомогти покращити час завантаження вашого веб-сайту. Мережі CDN створені для швидкої та ефективної доставки вмісту, і використовуючи CDN для обслуговування файлів CSS, ви можете допомогти скоротити час завантаження вашого веб-сайту та покращити взаємодію з користувачем.

Загалом використання Normalize.css може допомогти покращити узгодженість і якість ваших веб-проектів, а його використання через CDN може допомогти покращити час завантаження та продуктивність вашого веб-сайту.

ВПРАВИ

Вправи до цієї лекції знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers
https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua